The National Higher School of Artificial Intelligence
المدرسة الوطنية العليا للذكاء الاصطناعي

Course
« *Computer Vision*»

Sid-Ahmed Berrani

2025-2026

Image enhancement: Intensity transformation

- This part focuses on techniques that modify the intensity of pixels implemented in the **spatial domain** (i.e. the image plane containing the pixels of the image).
- A spatial domain process can be described by the expression:

$$g(x,y) = T[f(x,y)]$$

Output image

Operator on f defined over a neighborhood of (x,y)

Input image

Image enhancement: Intensity transformation

Typically, the neighborhood of (x,y) is:

- Rectangular
- Centered on (x,y)
- Much smaller than the size of the image

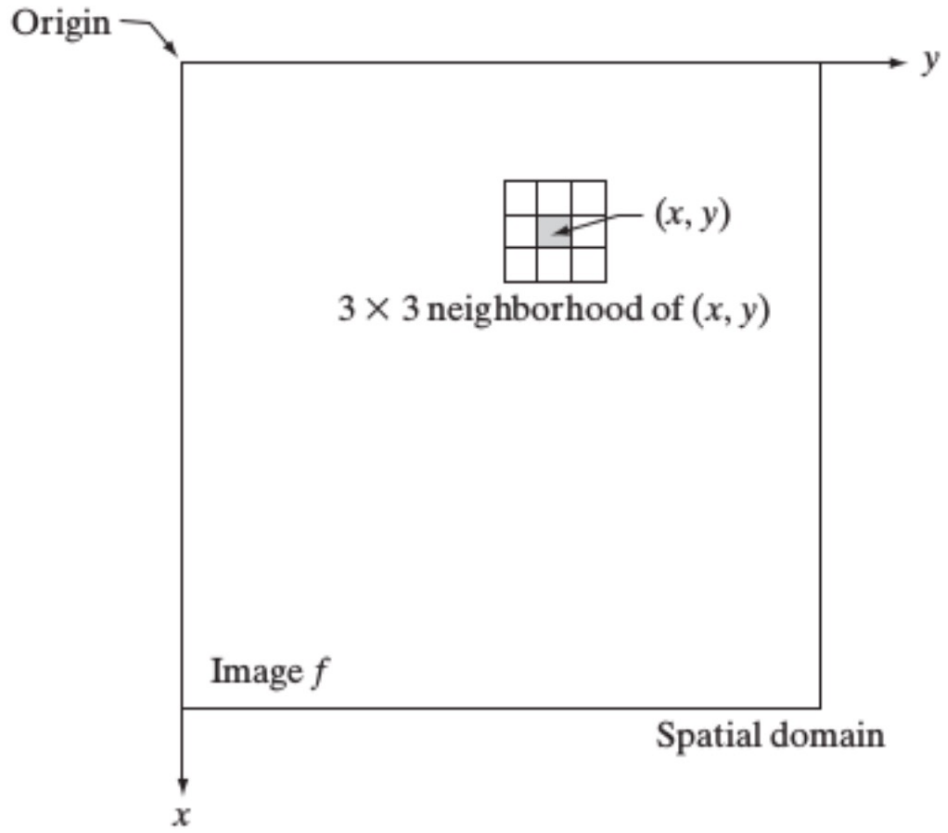


Image enhancement: Intensity transformation

- When the neighborhood has size 1x1, ***g(x,y)*** depends only on the value of ***f*** at ***(x,y)***
- ***T*** is an intensity transformation function:

$$s = T(r)$$

- Where ***s*** and ***r*** are the intensity of ***g()*** and ***f()*** at a generic point ***(x,y)***

Image enhancement: Intensity transformation

- **The negative**

The negative of an image with intensity levels in the range **[0…L-1]** is obtained by the following expression:

$$s = L - 1 - r$$

=> This processing enhances white or gray details embedded in dark regions

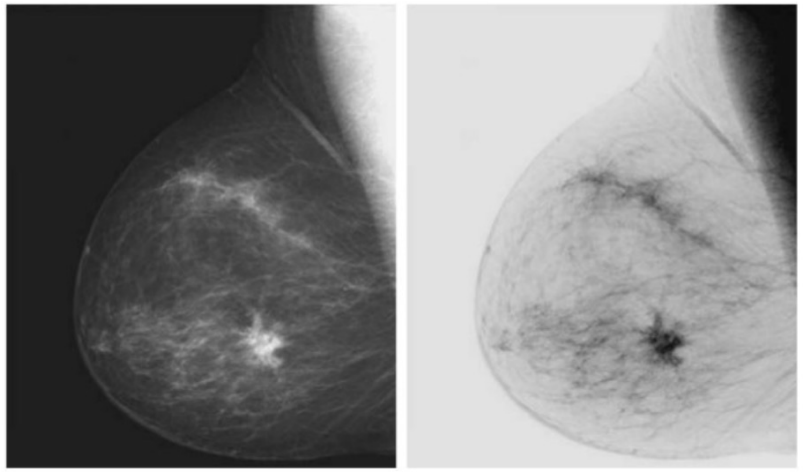


Image enhancement: Intensity transformation

- **Gain/Bias**

Two commonly used point processes are multiplication and addition with a constant:

$$s = \alpha\, r + \beta$$

The two parameters α>0 and β are often called *gain* and *bias.*

α controls **contrast**

β controls **brightness**

Image enhancement: Intensity transformation

- **Gain/Bias**

$$s = \alpha\, r + \beta$$

Original Image

$\alpha = 2$

$\beta = 100$

Image enhancement: Intensity transformation

- **Log Transformations**

Log transformations are useful to **compress the dynamic range** for images with large variation in pixel values.

It consists in replacing each pixel value with its logarithm.

$$\boldsymbol{s = c \cdot \log(1 + r)}$$

r: input pixel intensity (normalized, e.g. in $[0, 1]$)

s: output pixel intensity

c: scaling constant

The **+1** avoids $\log(0)$

$$c = \frac{S_{\max}}{\log(1 + R_{\max})}$$

Image enhancement: Intensity transformation

- **Log Transformations**

The value of **c** is chosen such that we get the maximum output value corresponding to the bit size used. e.g for 8 bit images.

That is, **c** is chosen such that we get max value equal to 255.

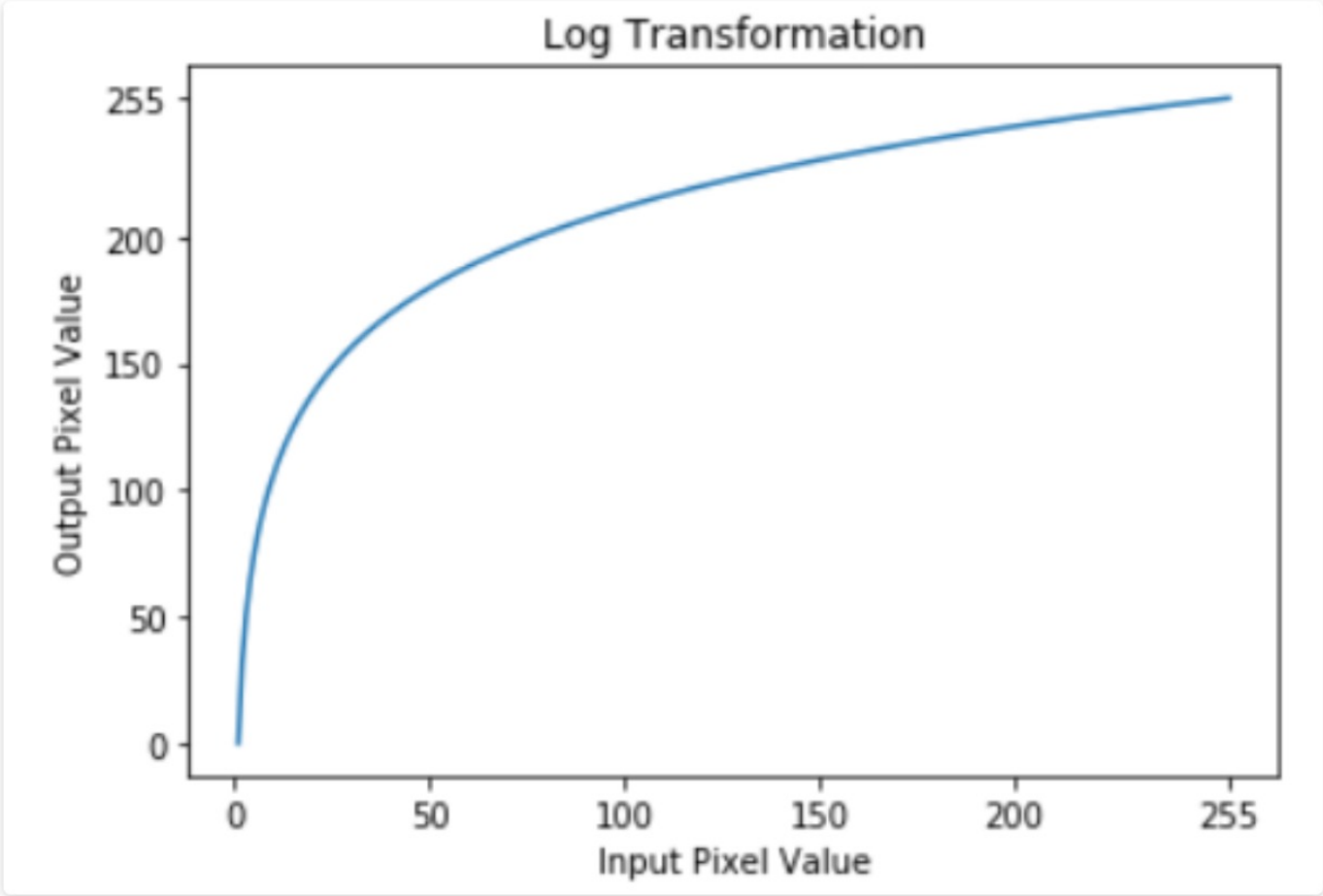


Image enhancement: Intensity transformation

- **Log Transformations**

⇒ Maps a wide range of high intensity values in input to a narrow range of output level

⇒ Large input values → **small output changes**

⇒ Bright pixels are **compressed**

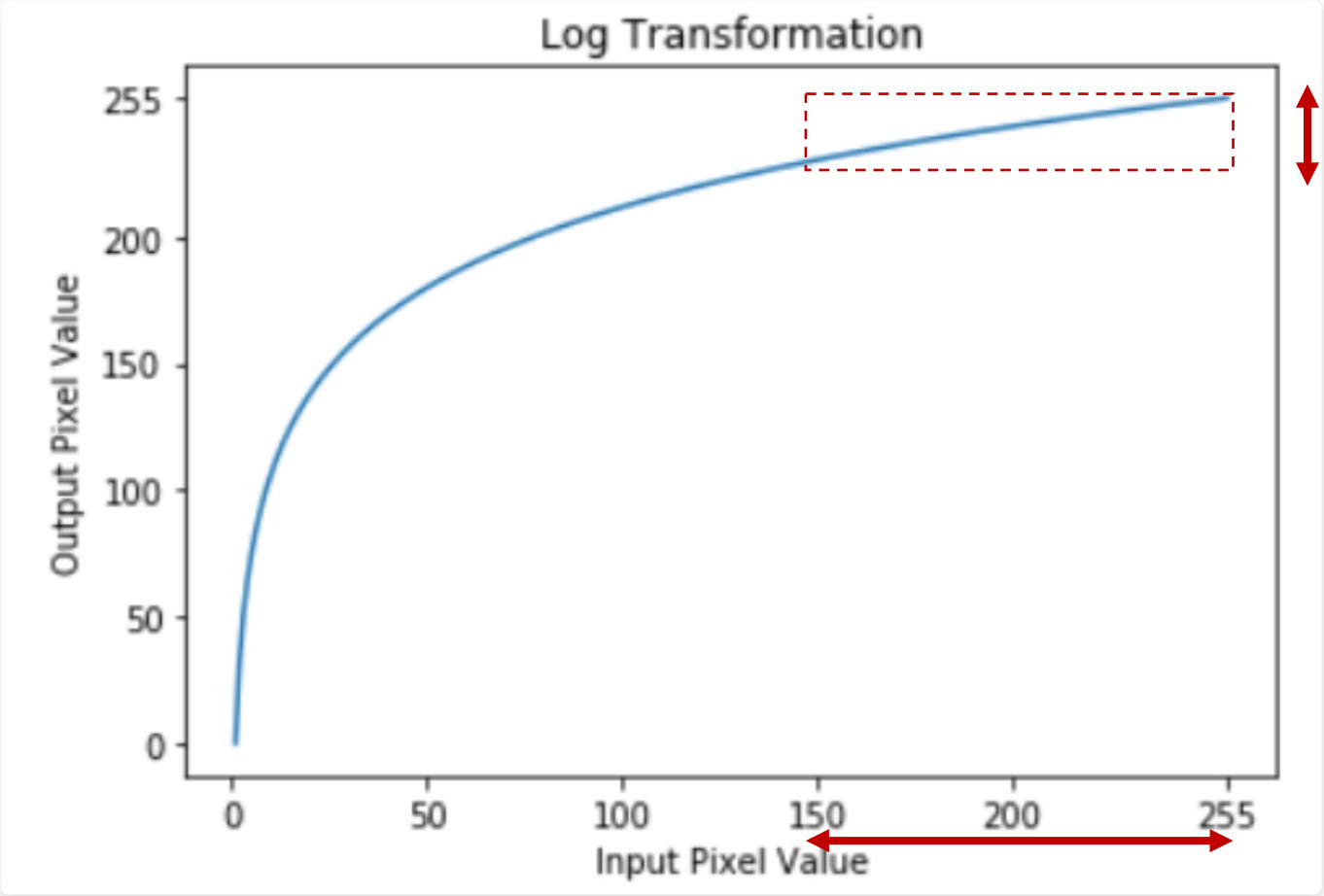


Image enhancement: Intensity transformation

- **Log Transformations**

⇒ low intensity values in the input image are mapped to a wider range of output levels.

⇒ Small input values → **large output changes**

⇒ Dark pixels are **stretched**

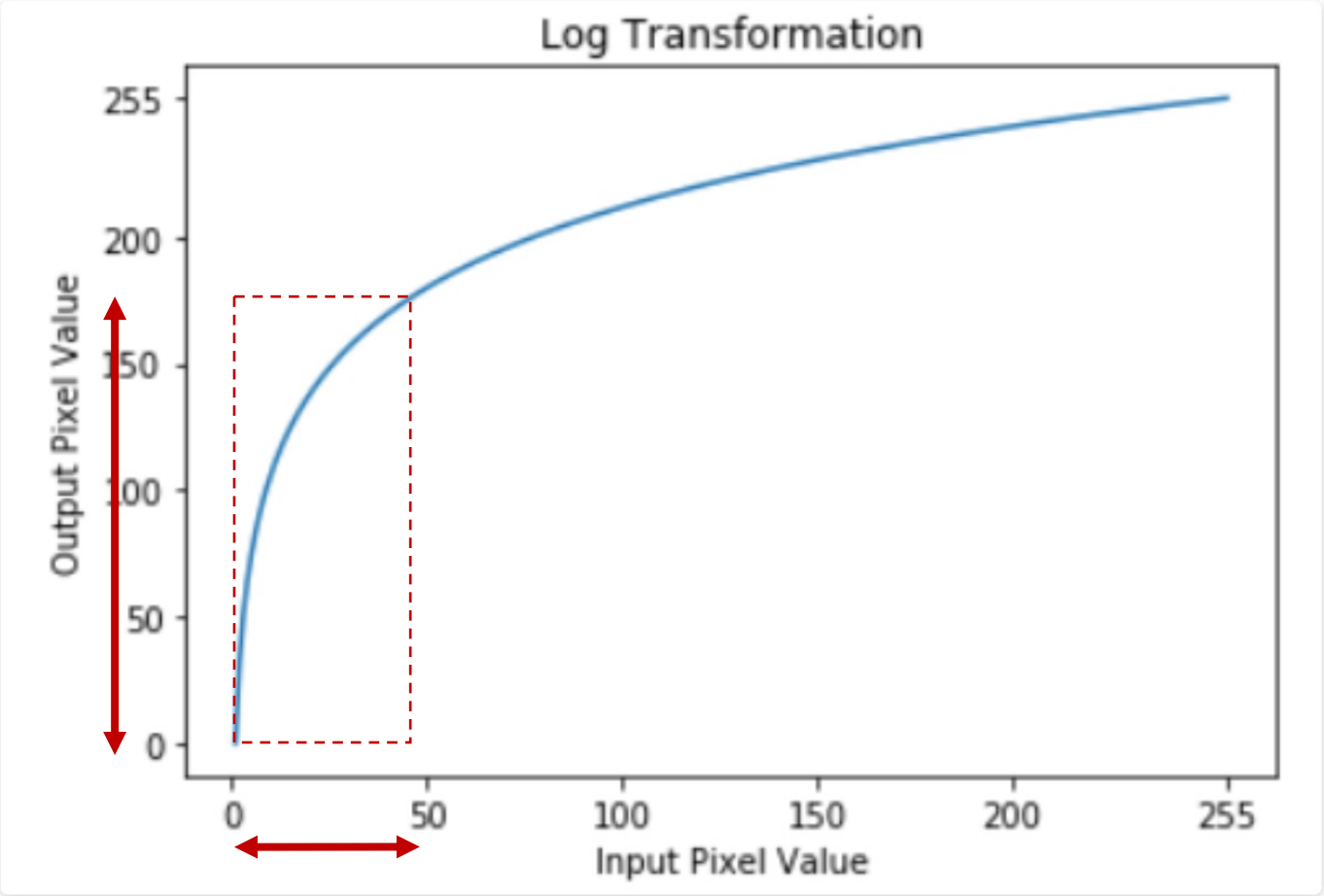


Image enhancement: Intensity transformation

Log Transformations: Visual interpretation on an image

- Before log transform:
  - Dark regions are almost black
  - Details are hidden
- After log transform:
  - Dark regions become brighter
  - Subtle variations become visible
  - Bright regions change little
- This is why log transform is often used to **reveal hidden details**.

Image enhancement: Intensity transformation

- **Log Transformations: Applications**

In other terms, a logarithmic transform is appropriate when we want to enhance the low pixel values at the expense of loss of information in the high pixel values.

Image enhancement: Intensity transformation

- **Log Transformations : Applications**

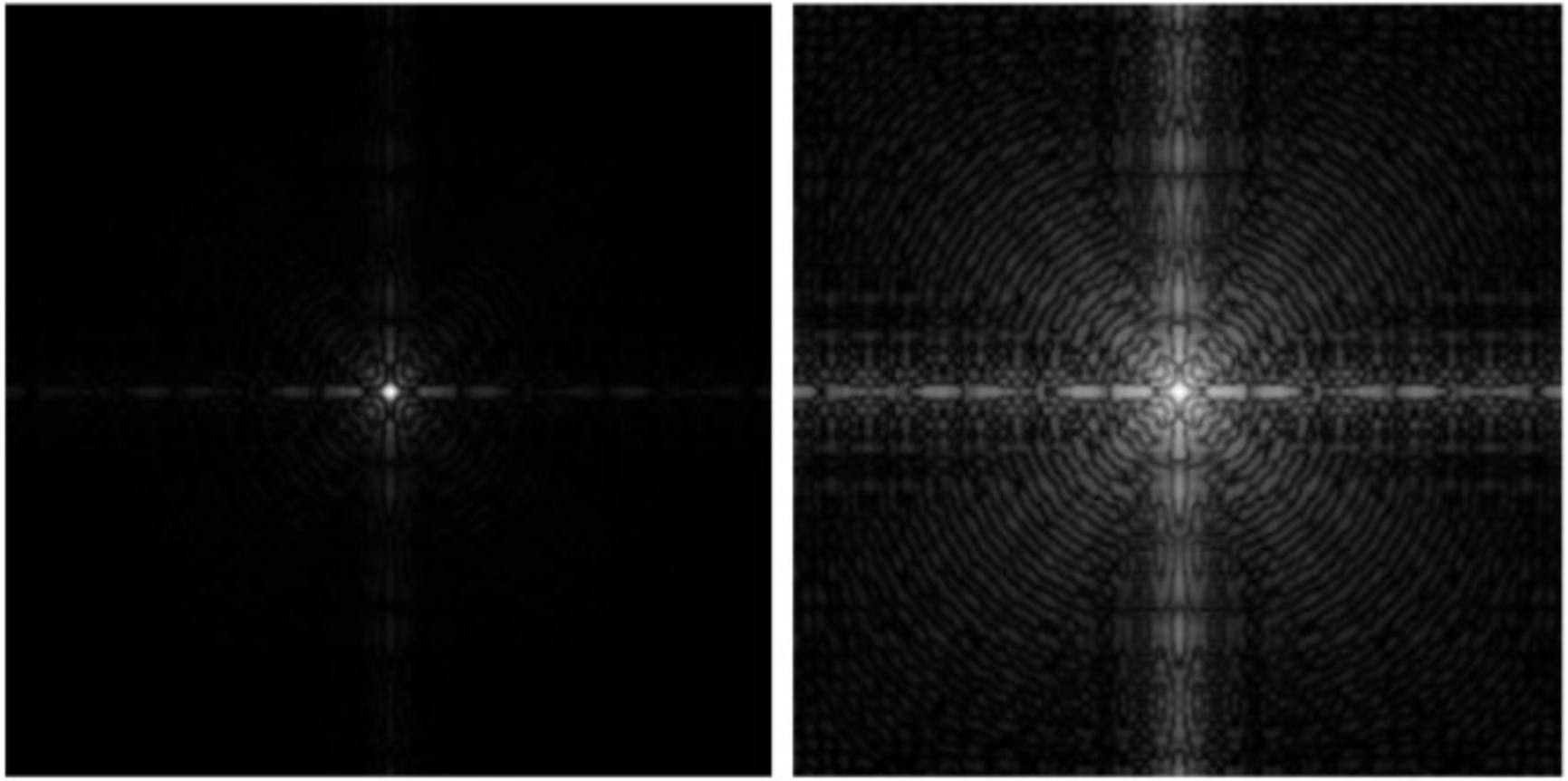


Image enhancement: Intensity transformation

- **Log Transformations: Applications**

But be careful if most of the details are present in the high pixel values…

Before

After

Image enhancement: Intensity transformation

- **Gamma (or power law) transformations**

They have the following basic form: $\boldsymbol{s = c.r^{\gamma}}$

where $\boldsymbol{c}$ and $\boldsymbol{\gamma}$ are positive constants.

- Compress values similar to log transformation but more flexible due to the $\boldsymbol{\gamma}$ parameter
- Curves generated with a $\boldsymbol{\gamma}$>1 have the opposite
- effect of those with $\boldsymbol{\gamma}$<1
- Identity transformation when **c=γ=1**

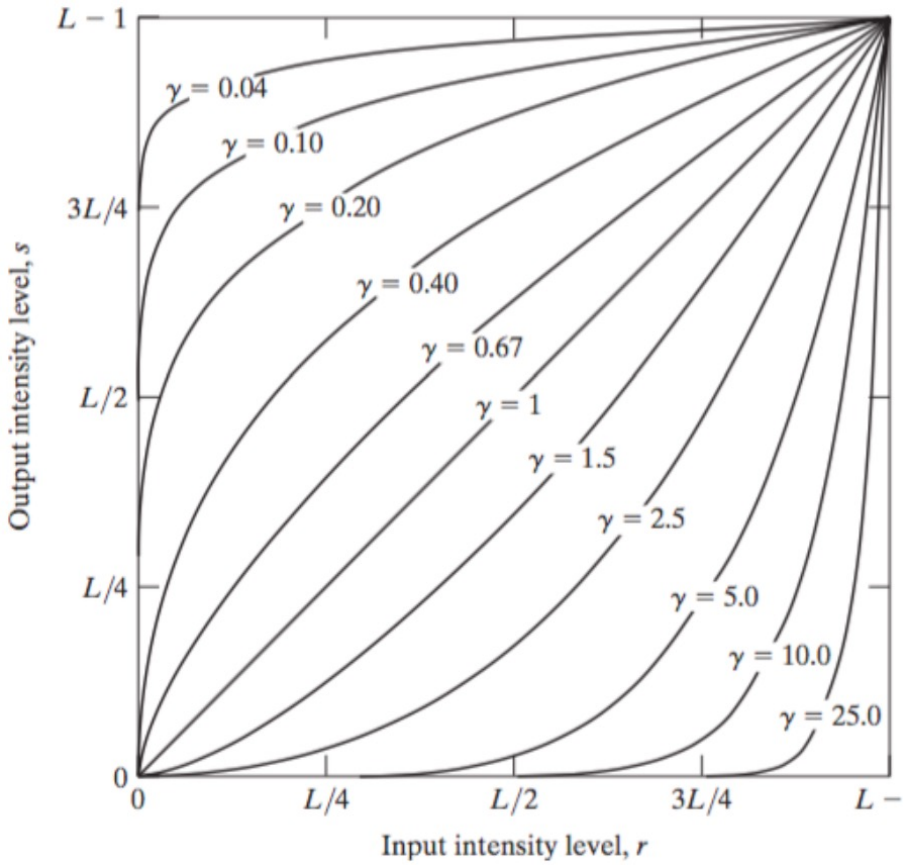


Image enhancement: Intensity transformation

What problem gamma correction solves

- Displays (CRT historically, and effectively modern displays too) are **not linear**.
- If you send a pixel value v to a display, the **emitted luminance** is roughly:

$$L \propto v^{\gamma display} \text{ with } \gamma_{display} \approx 2.2$$

- A pixel value of 128 does **not** produce half the brightness of 255
- Dark values are **compressed**
- Bright values are **expanded**

This is a **hardware non-linearity**.

Image enhancement: Intensity transformation

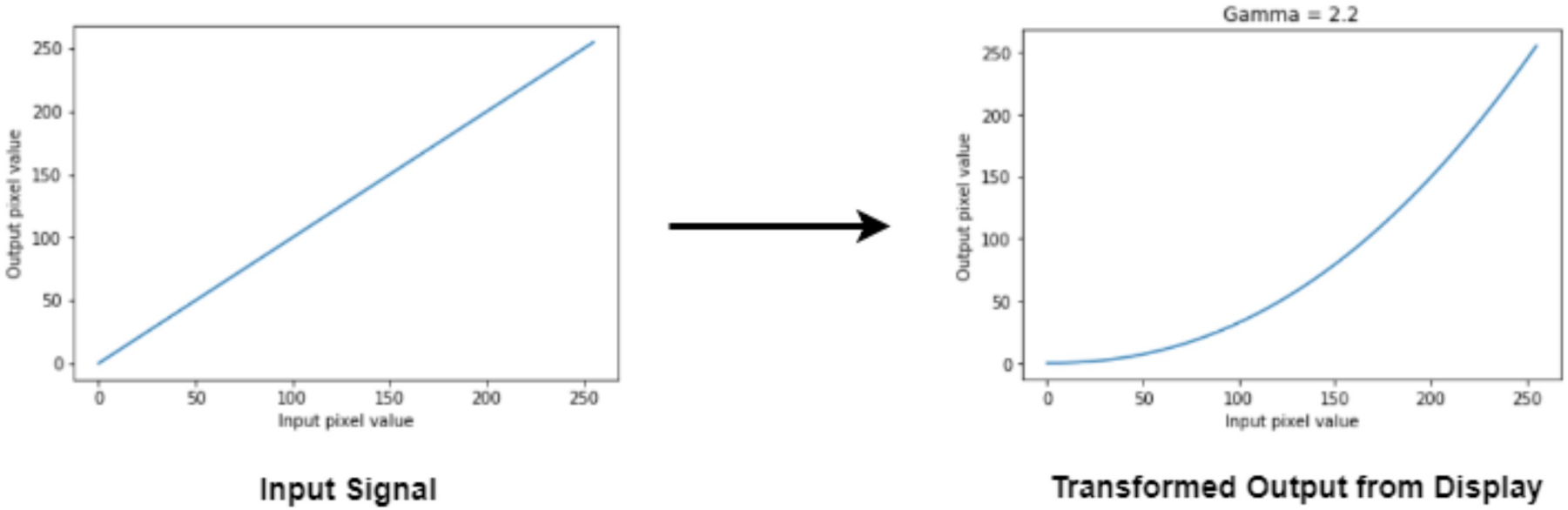
- **Gamma (or power law) transformations**

Gamma correction is useful because many image capture/printing/display devices have a power law response (not linear!).

- For example, old CRT monitors or modern projectors have an intensity-to-voltage response which is a power-law with exponents varying from 1.8 to 2.5
- By using gamma correction we can remove this effect to obtain a response that is similar to the original image

Image enhancement: Intensity transformation

- **Gamma (or power law) transformations**



Input Signal **Transformed Output from Display**

- So if you send a **linear image** to the display => It will appear darker than intended.

Image enhancement: Intensity transformation

- **Gamma (or power law) transformations**

To correct this, we apply gamma correction to the input signal

We know the intensity and voltage relationship => we simply take the complement

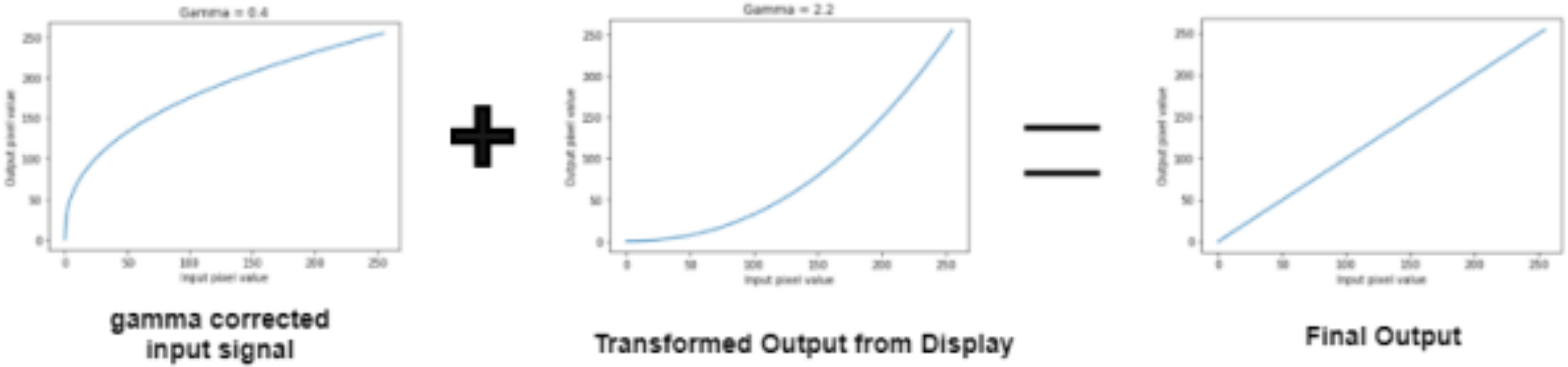
=> which is known as Image Gamma.

This gamma is automatically applied by the conversion algorithms like jpeg etc. thus the image looks normal to us.

This input cancels out the effects generated by the display and we see the image as it is. The whole procedure can be summed up as by the following figure

Image enhancement: Intensity transformation

- **Gamma (or power law) transformations**



⇒ **Gamma correction pre-distorts pixel values so that the non-linear display produces a linear perceived brightness.**

Image enhancement: Intensity transformation

- **Gamma (or power law) transformations**

Original Image

Image enhancement: Intensity transformation

- **Gamma (or power law) transformations**

Image is pre-processed by applying a gamma transformation with $\gamma = 1/(2.5) = 0.4$

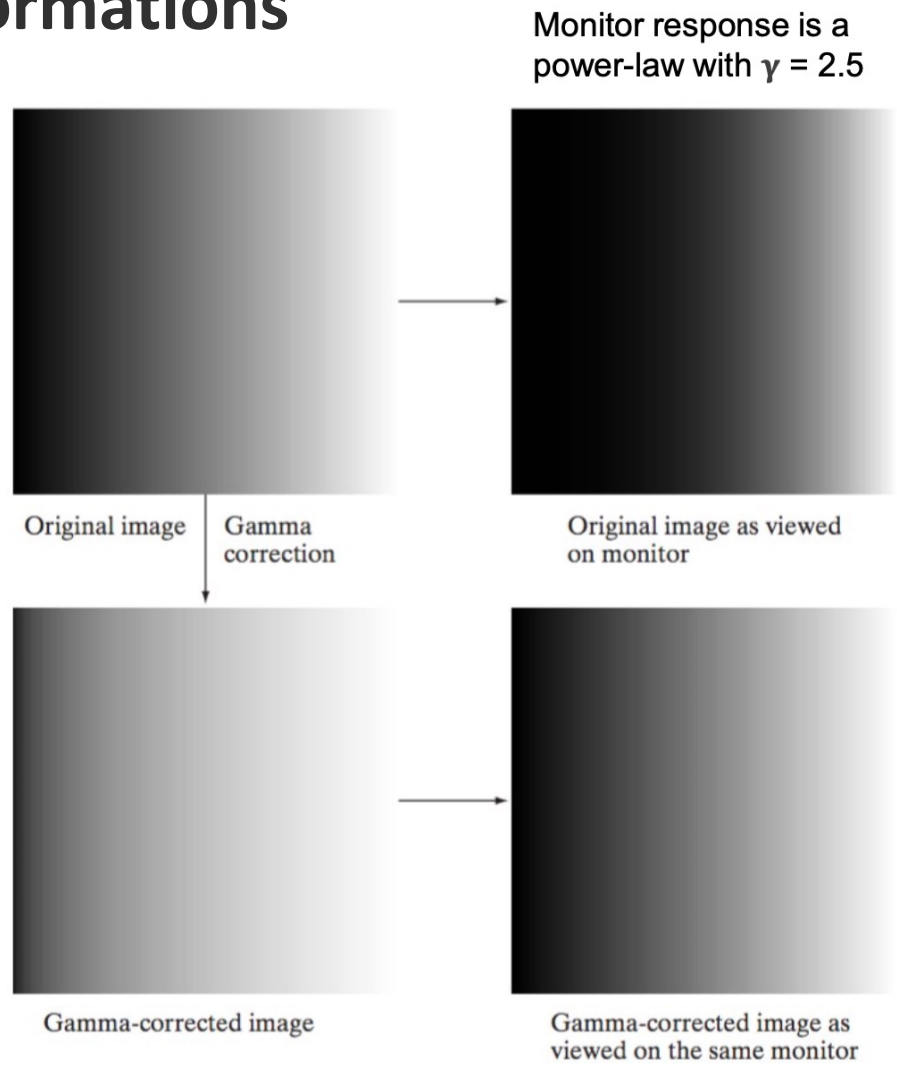


Image enhancement: Intensity transformation

- **Gamma (or power law) transformations**

=> Also useful for

contrast manipulation.

Image enhancement: Intensity transformation

- **Contrast Enhancement**

A whole family of transformations are defined using piecewise-linear functions.

One of the simplest and most useful piecewise-linear transformation is contrast enhancement (with a sigmoid shape), also called **Contrast-Stretching.**

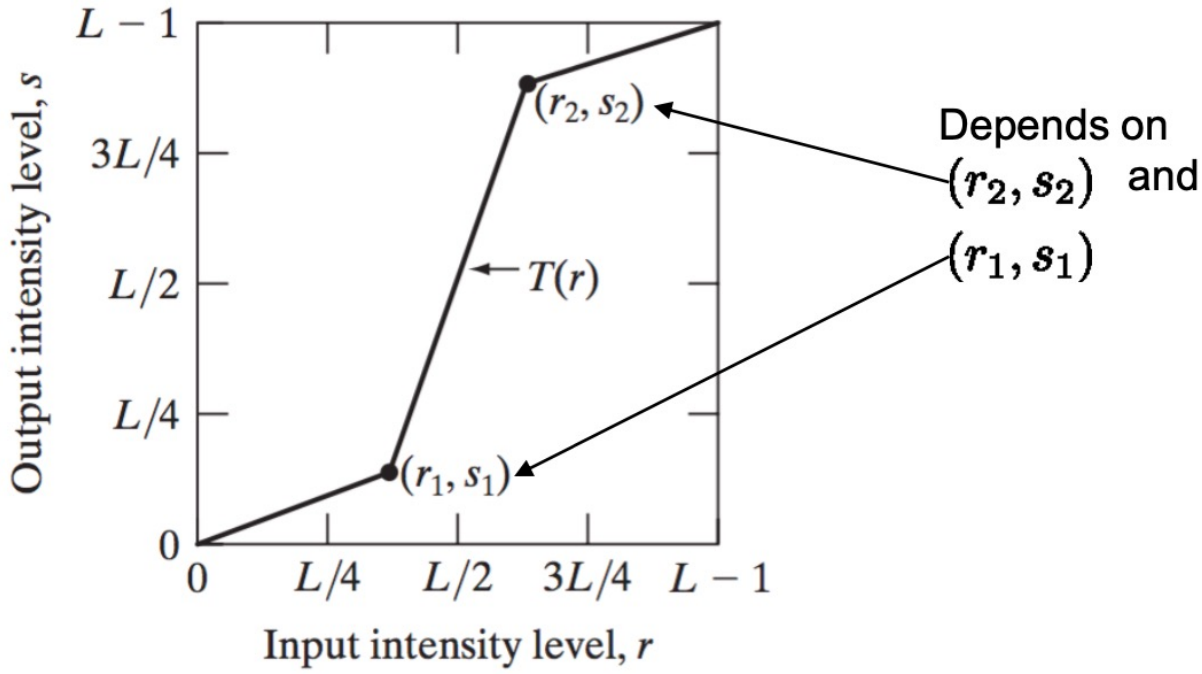


Image enhancement: Intensity transformation

- **Contrast Enhancement**

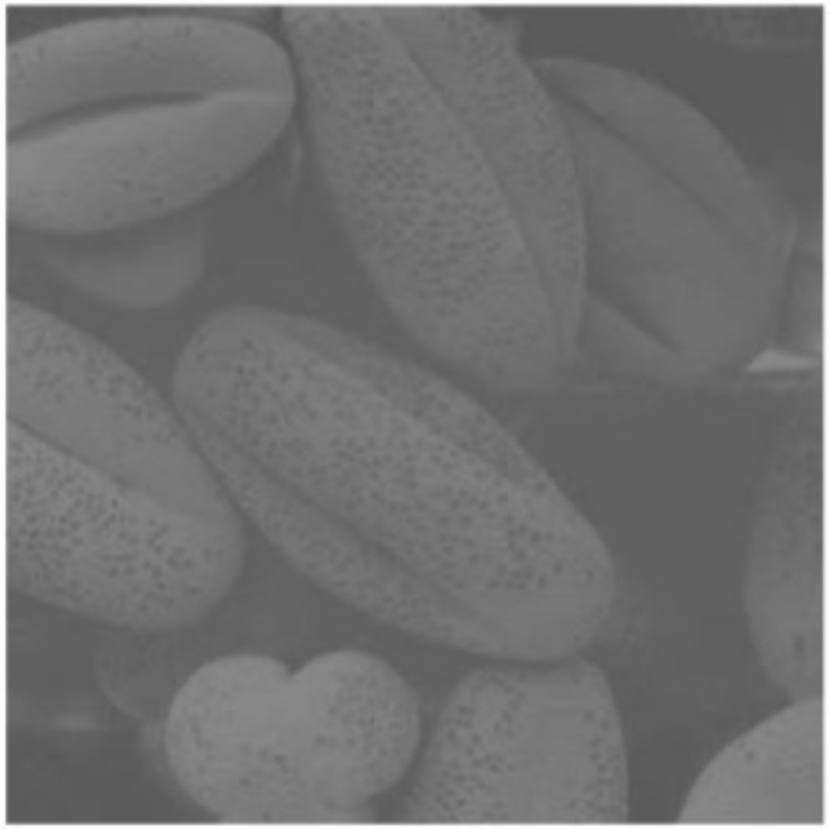
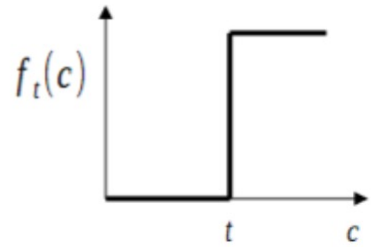


Image enhancement: Intensity transformation

- **Thresholding**

An extreme case of contrast enhancement is:

$$s = \begin{cases} 0 & \text{if } r \leq t \\ 1 & \text{if } r > t \end{cases}$$



Where ***t*** is a constant defined for the whole image.
If ***t*** depends on the spatial coordinates it is often referred as adaptive thresholding

Image enhancement: Intensity transformation

All the functions described so far can improve the appearance of an image by varying some parameters

=> How can we automatically determine the best values of these parameters?

- One effective tool is the **Image Histogram**
- It allows us to analyze problems in the intensity (or color) distribution of an image.
- Without spatial information, we can assimilate I(x,y) as a random intensity emitter.
- ***The image histogram is the empirical distribution of image intensities.***

Image enhancement: Intensity transformation

- **Image Histogram**

Let [0 … L-1] be the intensity levels of an image. The image histogram is a discrete function:

$$h(r_k) = n_k$$

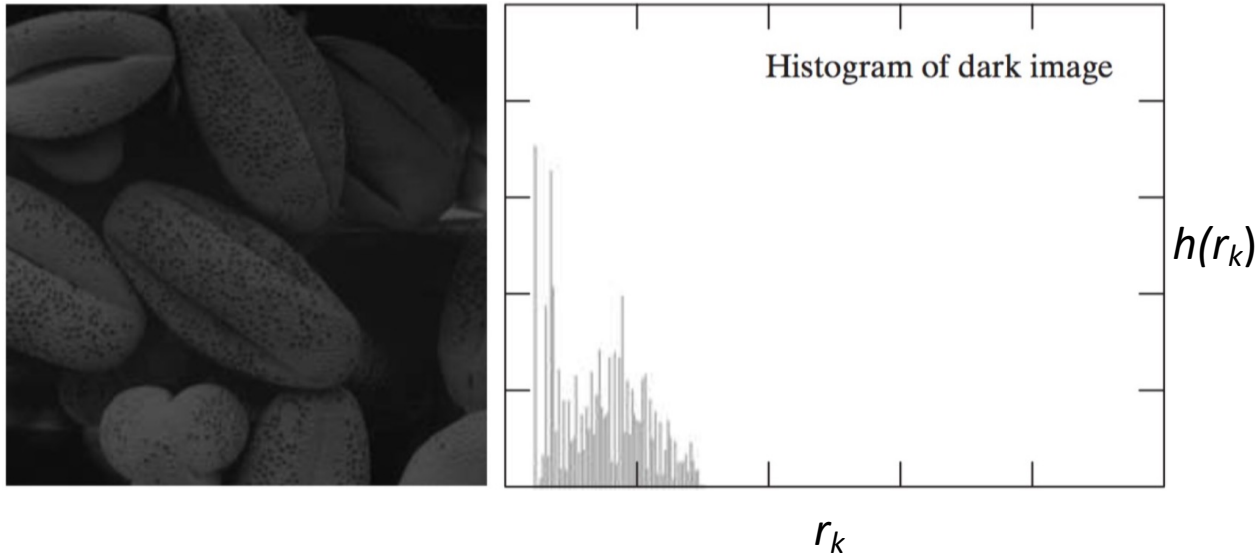
where r_k is the k^{th} intensity value and n_k is the number of pixels in the image with intensity r_k.

Usually the histogram is normalized by dividing each component to the total number of pixels

$\Rightarrow$ *each histogram component is an estimate of the probability of the occurrence of the intensity* r_k

Image enhancement: Intensity transformation

- **Image Histogram** $h(r_k) = n_k$



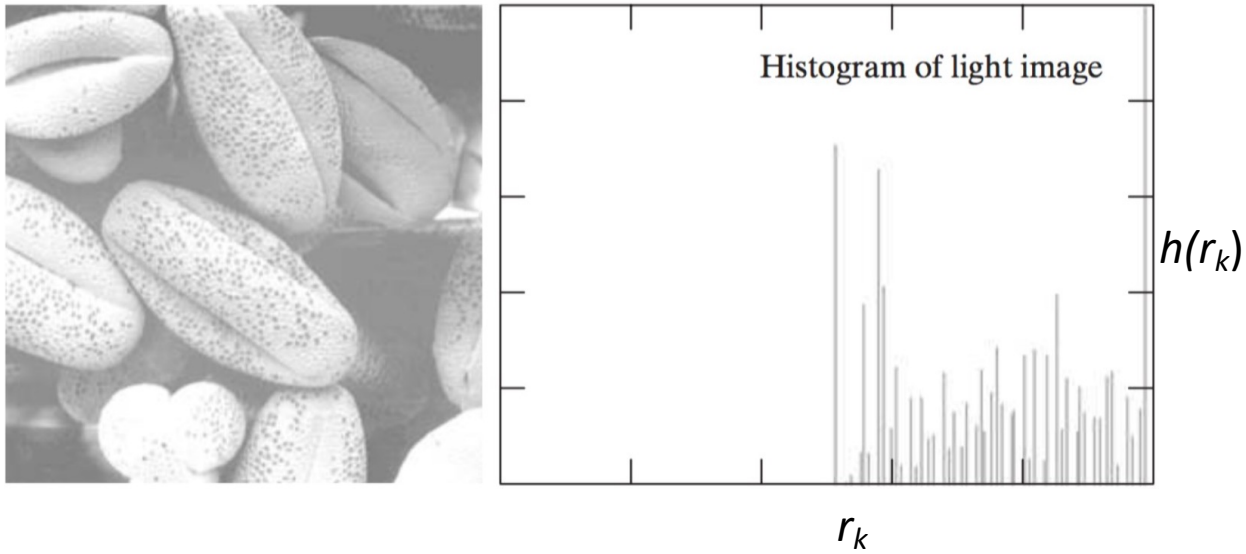


Image enhancement: Intensity transformation

- **Image Histogram** $h(r_k) = n_k$

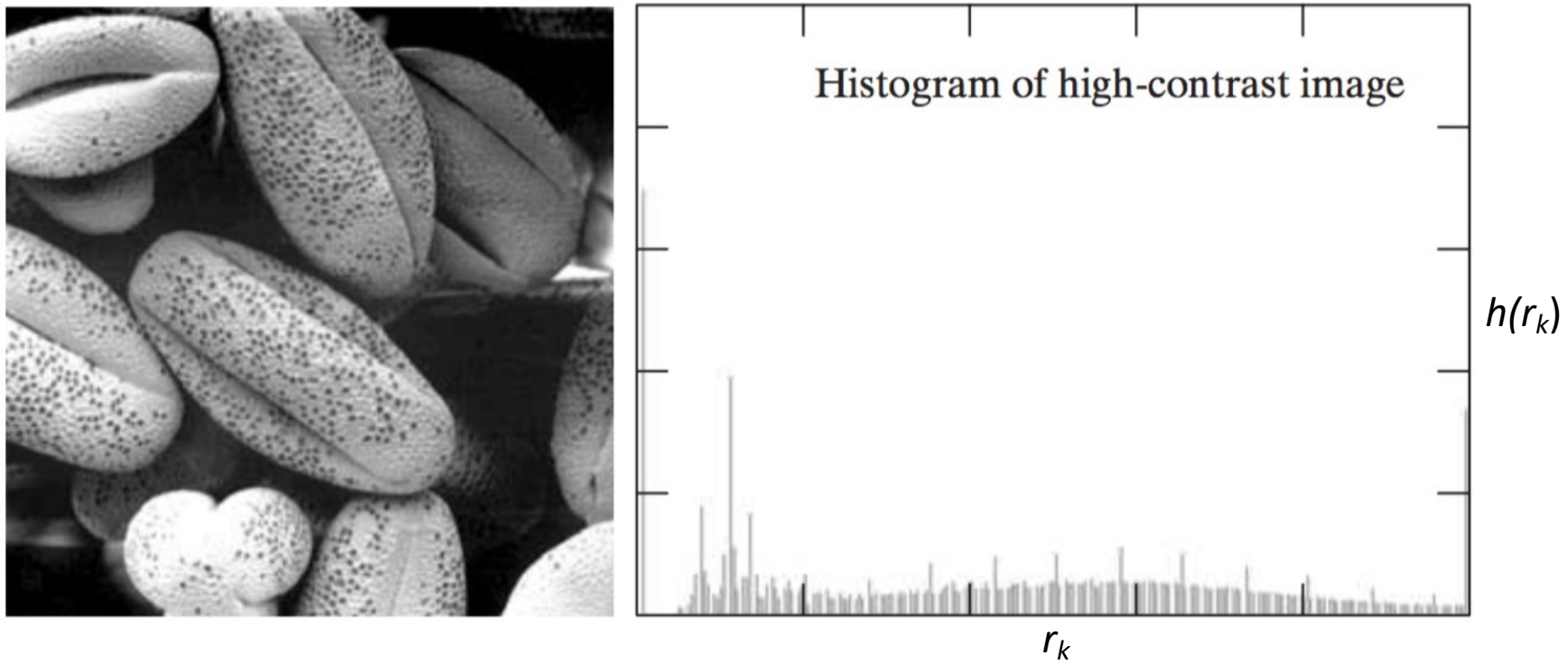
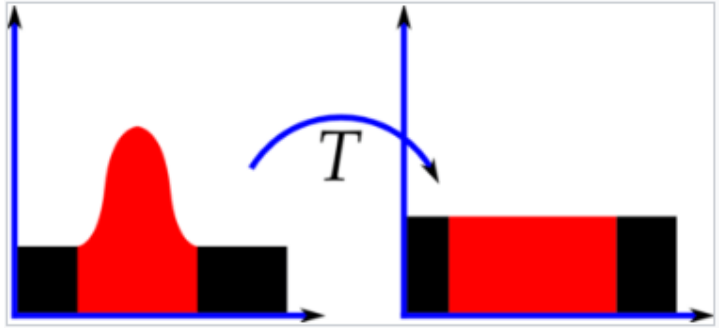


Image enhancement: Intensity transformation

Histogram equalization

As the name suggests, stretches the histogram to fill the dynamic range and at the same time tries to keep the histogram uniform.



$$s = T(r)$$

⇒ When enhancing an image, ideally we would like to maximize the dynamic range of the image to catch both the dark and bright details.

Image enhancement: Intensity transformation

Histogram equalization

- **Input intensities**

  r_k= the **k-th gray level** in the original image; $k = 0,1, \dots, L-1$

- **Output intensities**

  s_k= the **new gray level** after equalization.

Step 1: Historgam of input intensities $n_k =$ *number of pixels having intensity* r_k

Step 2: Convert histogram to probabilities $p_r(r_k) = \frac{n_k}{MN}$. $M \times N$= total number of pixels

$p_r(r_k)$= probability that a randomly chosen pixel has intensity r_k.

Step 3: Compute cumulative distribution (CDF) $T(r_k) = \sum_{j=0}^{k} p_r(r_j)$. (fraction of pixels with intensity ≤ r_k)

Image enhancement: Intensity transformation

Histogram equalization

Step 4 : Compute new intensity values $\boxed{s_k = (L-1)\,T(r_k)}$

$$s_k = (L-1)\sum_{j=0}^{k}\frac{n_j}{MN}. \quad \Rightarrow$$

If *pixel value* = r_k $\Rightarrow$ *replace it with* s_k

Why does this improve contrast?

- Because the mapping spreads pixel values across the full range:
- Dark levels move toward lower intensities
- Bright levels move toward higher intensities
- Middle levels get redistributed

  => The histogram becomes more uniform

  => The dynamic range is better used

Image enhancement: Intensity transformation

Histogram equalization

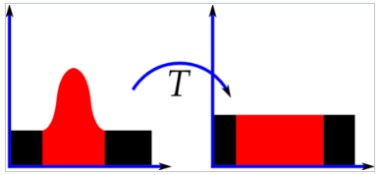
Why does histogram equalization work? Why does using the CDF make the output histogram uniform?

=> The key idea: We treat pixel intensities as random variables:

r= original gray level (input). $p_r(r)$= PDF of the input image.

s= transformed gray level (output). $p_s(s)$= PDF after transformation.

The objective : $\boxed{p_s(s) = constant}$



=> Transform intensities $s = T(r)$

The number of pixels that were in a small range around $r$must be the same as those mapped around s

(we do not create or destroy pixels)

Image enhancement: Intensity transformation

Histogram equalization

- Let's consider a small intensity interval: $[\boldsymbol{r}, \boldsymbol{dr}]$ => It contains $\boldsymbol{p_r(r)\, dr}$ fraction of pixels.
- After transformation, the interval becomes: $[\boldsymbol{s}, \boldsymbol{ds}]$ => It contains $\boldsymbol{p_s(s)\, ds}$ fraction of pixels.
- Since the number of pixels is preserved (the **density conservation law**): $\boxed{\boldsymbol{p_r(r)\, dr = p_s(s)\, ds}}$
- $p_s(s) = p_r(r) \left|\frac{dr}{ds}\right|$ => Output density depends on input density and how much we stretch/compress the intensities.
- If we want: $p_s(s) = 1$ => $\frac{ds}{dr} = p_r(r)$ *(here we consider $s \in [0,1]$)*
- $s = \int_0^r p_r(w)\, dw$
- This is exactly: $\boxed{s = CDF(r)}$

Image enhancement: Intensity transformation

Histogram equalization

Histogram equalization works because:

- It **maps pixel values according to their cumulative rank** in the image.

- Crowded intensity regions get stretched
- Sparse regions get compressed

Result:

- Pixel density becomes uniform.

Image enhancement: Intensity transformation

Histogram equalization

Intensity levels of an image may be viewed as random variables in interval [0 … L-1].

Image histogram of *s* is an estimate of the PDF of *s* ($p_s(s)$) and histogram of *r* is an estimate of $p_r(r)$.

From probability theory, when we apply a function $s = T(r)$ to a random variable r we got:

$$p_s(s) = p_r(r)\left|\frac{dr}{ds}\right|$$

(T must be continuous, differentiable and strictly monotonic)

Image enhancement: Intensity transformation

Histogram equalization

Cumulative distrib. Function of *r*

The used function *(T)* is the following: $$T(r) = (L-1)\int_0^r p_r(w)dw$$

where *L* is the maximum intensity value (for *n* bit image, $L = 2^n$)

=> $$p_s(s) = \frac{1}{L-1}$$ Uniform distribution

In the discrete case: $$s_k = T(r_k) = (L-1)\sum_{j=0}^{k} p_r(r_j) = \frac{L-1}{MN}\sum_{j=0}^{k} h(r_j)$$

Sum of the first *k* components of the input image histogram

=> We obtain a **(quasi)** flat histogram of the output image.

Image enhancement: Intensity transformation

Histogram equalization

Since histogram is a discrete approximation of a PDF, the resulting histogram is in general not perfectly flat. It still remains a good approximation!

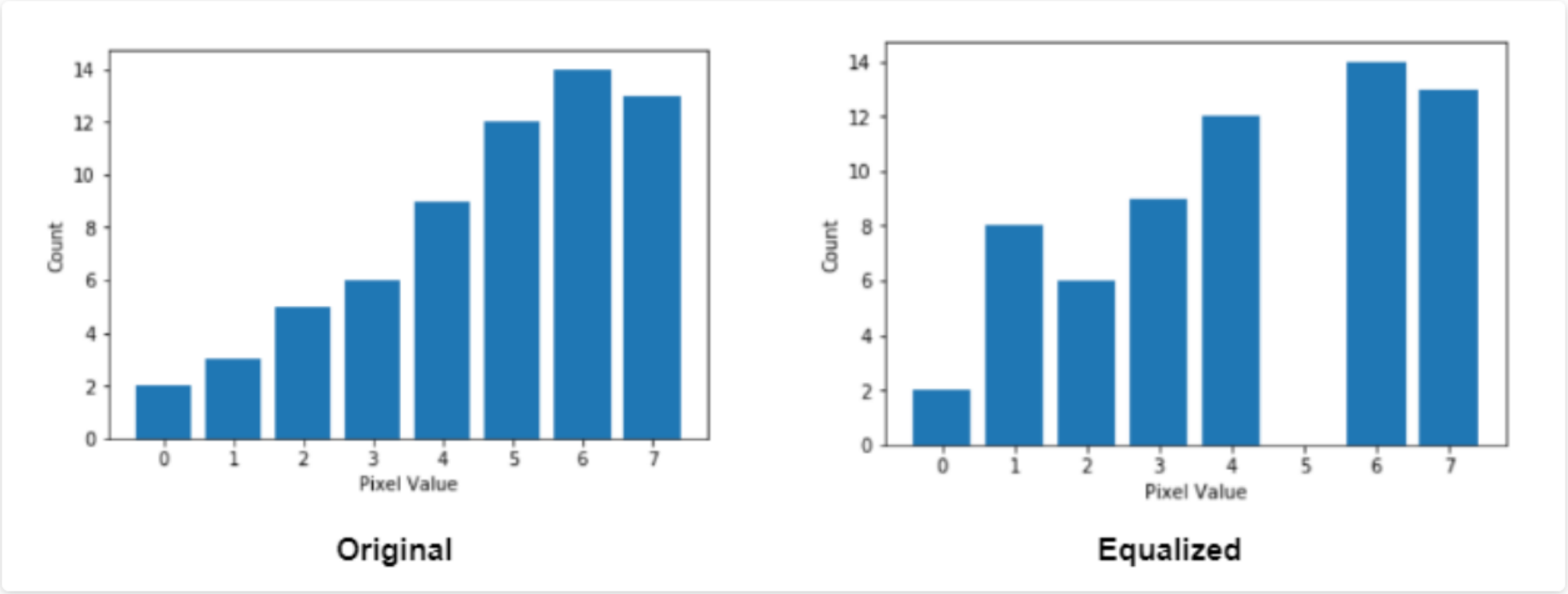


Image enhancement: Intensity transformation

Histogram equalization

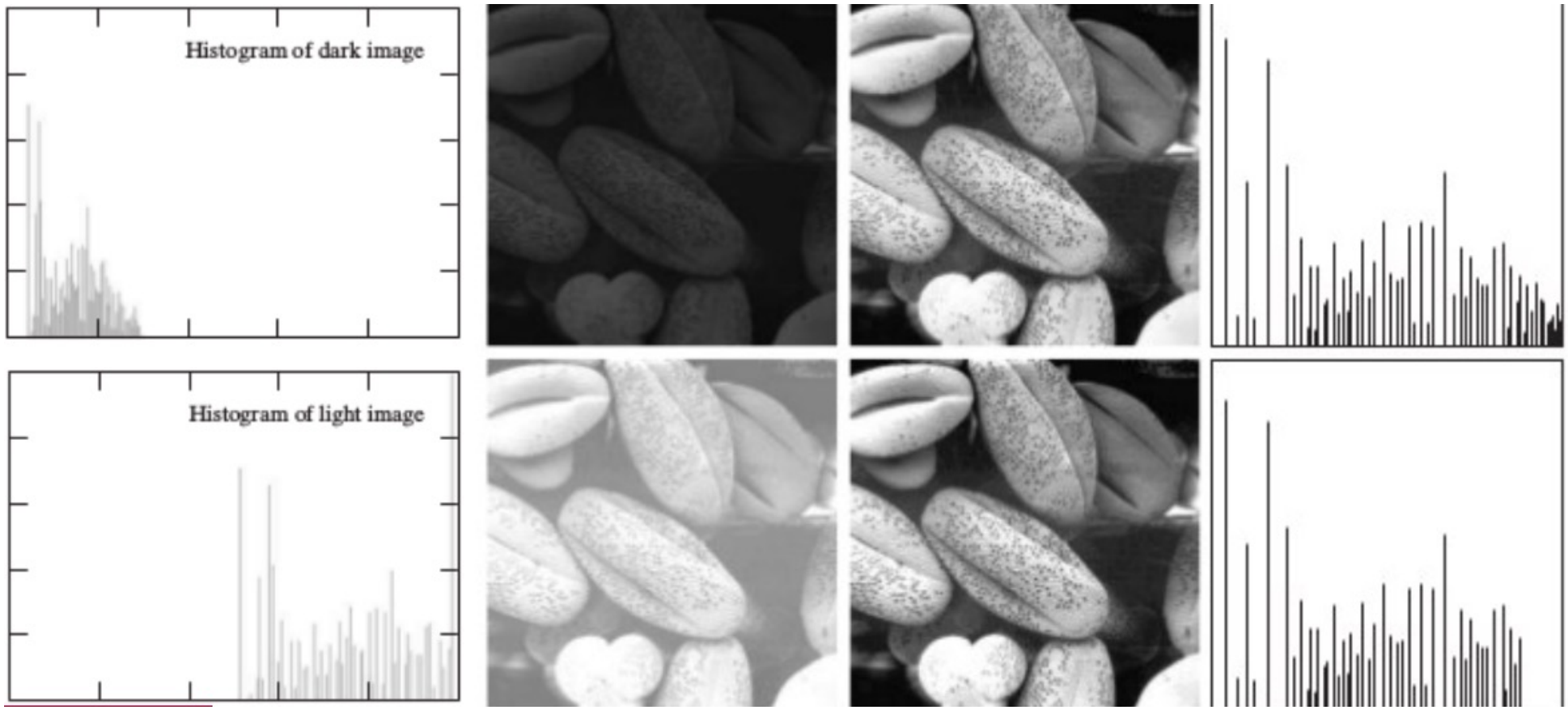


Image enhancement: Intensity transformation

Histogram equalization

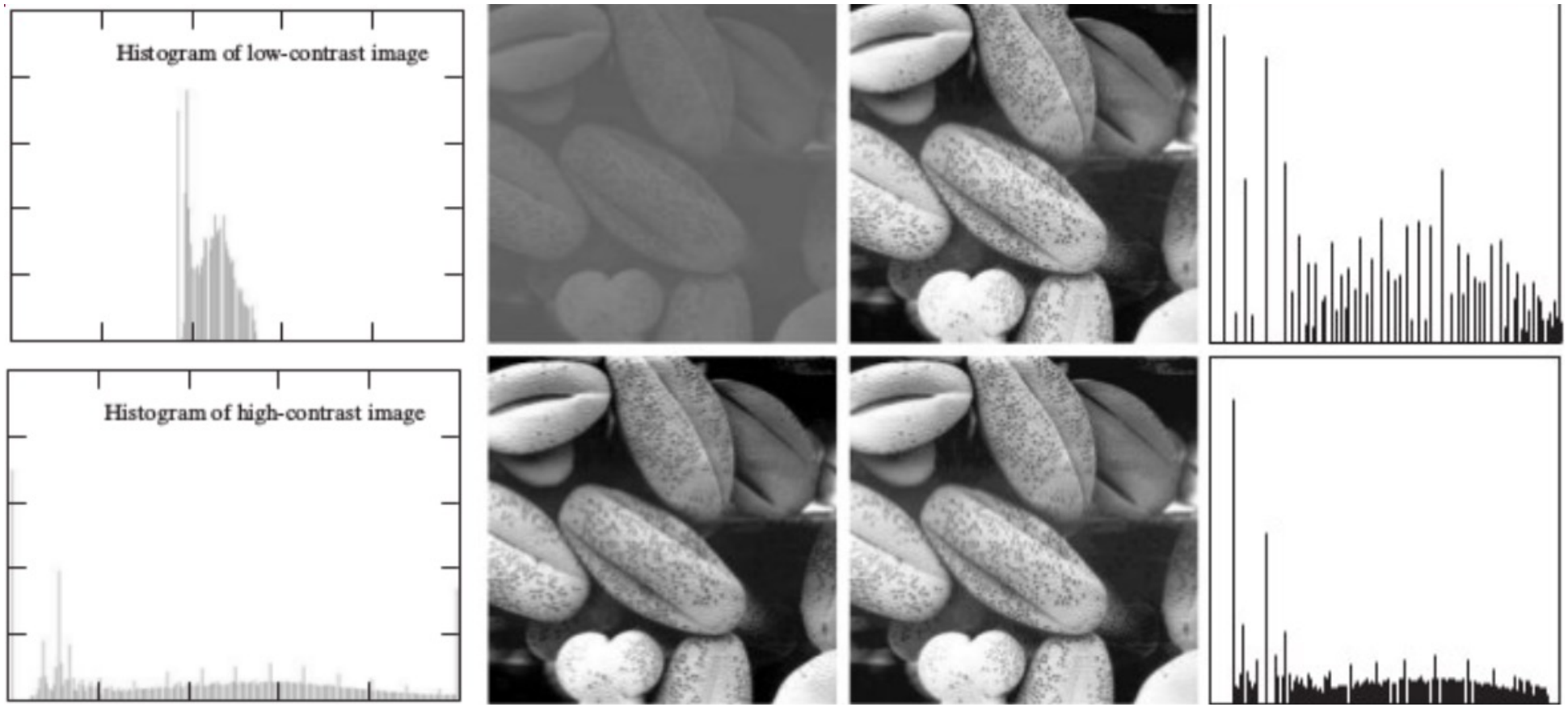
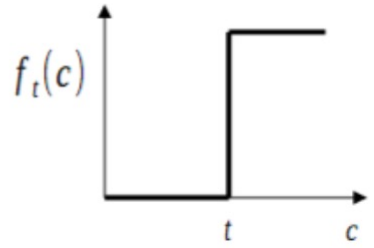


Image enhancement: Intensity transformation

Histogram for thresholding

$$s = \begin{cases} 0 & \text{if } r \leq t \\ 1 & \text{if } r > t \end{cases}$$



When using global thresholding a common problem is to automatically find a good threshold ***t*** that separates well dark from bright areas.

Image enhancement: Intensity transformation

Histogram for thresholding

Image histogram can give us useful clues on the threshold level.

If an image is separable through thresholding there will be a range of intensity with low probability.

Thresholding is essentially a clustering problem in which two clusters (black and white pixels) are sought.

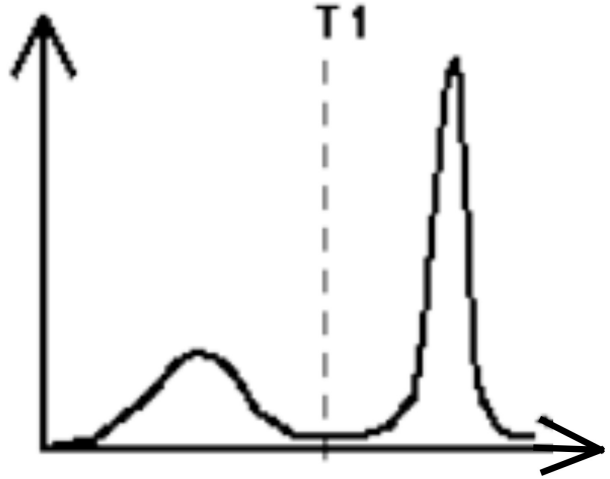
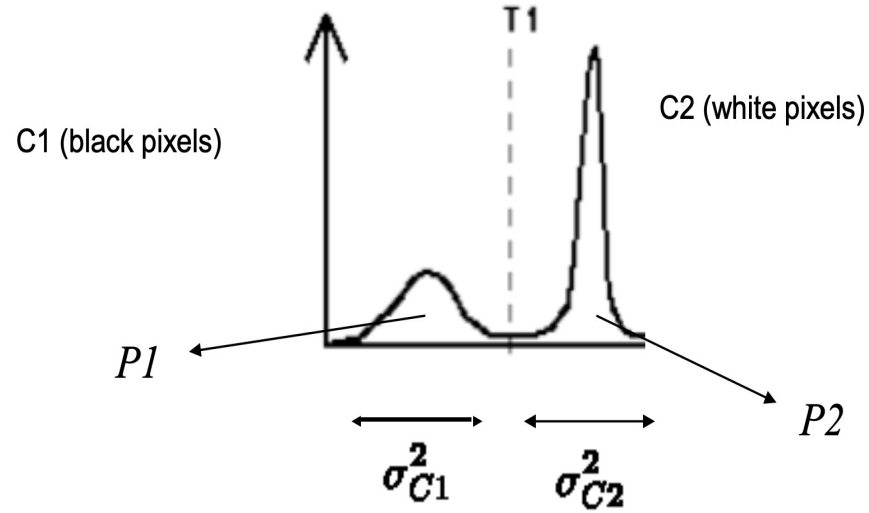


Image enhancement: Intensity transformation

Otsu Thresholding

The idea: find the threshold that minimizes the weighted sum of the within-class variances of the 2 classes.:

$$\arg\min_{T}\left(P_1\sigma_{C1}^2 + P_2\sigma_{C2}^2\right)$$



- C1 = pixels with intensity ≤ T (dark class)
- C2 = pixels with intensity > T (bright class)
- P_1 = fraction of pixels in class C1
- P_2 = fraction of pixels in class C2

The best threshold is the one that makes each class as “compact” as possible.